
Platform Primitives

Canonical Constitution

AI Workspace Platform · Product Philosophy Document

Classification	Product Philosophy / Conceptual Systems Design
Scope	Core entity model, interaction grammar, surface allocation, emotional architecture
Status	Canonical — changes require constitutional justification, not feature rationale
Version	1.0

This document is a constitution, not a backlog. It defines what the platform is, not what it will ship next. Amendments require structural justification, not feature rationale.

THE GOVERNING TEST

A primitive earns its existence by failing the removal test.

If the product does not fundamentally break — structurally, cognitively, or emotionally — without a given concept, that concept is a feature, not a primitive. Features can be deferred, versioned, or cut.

Primitives cannot. They are the product's skeleton. Every design decision that touches a primitive must be evaluated against conceptual integrity, not just usability or aesthetic preference.

A second test governs emotional primitives specifically: does the absence of this primitive make the user feel like they're using a chatbot? If yes, it is a primitive. The product's identity is defined as much by what it refuses to be as by what it is.

PART I

The Entity Model

Entities are the nouns of the platform. They exist, persist, and have state. The entity model has three tiers, organised by what fails without them.

TIER 1 — FOUNDATIONAL

The system cannot exist without these. They are the vocabulary of the workspace.

Project

WHAT IT IS

A bounded context of ongoing work. A Project has a goal, a history, an active memory state, and a set of agents assigned to it. It is not a folder, a repository, or a conversation thread. It is the unit of directed intelligence.

WHY IT EXISTS

Without the Project as an explicit, persistent entity, the workspace degrades into a flat sequence of interactions. There is no container for memory, no scope for agent context, no unit of focus. The user cannot return to something.

EMOTIONAL ROLE

Identity and ownership. The Project is the answer to 'what am I building?' It grounds the user. It is the place you return to. When you open a Project, the workspace should feel like entering a room where your work is already in progress.

WHAT BREAKS WITHOUT IT

Memory has no scope. Agents have no assignment. The thread has no container. Diverge and Convergence have no home. The workspace becomes a chat interface with extra steps.

User-visible	Always — a Project is never implicit.
Persistent	Yes — indefinitely, until explicitly archived.
Auditable	Yes — creation, membership changes, goal revisions, archival.

Agent

WHAT IT IS

A named, configured intelligence with a defined role, a set of capabilities, a tool access list, and a memory scope. An Agent is not a model. It is not a persona. It is a specialist. You name it, define its role, and trust it with a bounded domain of work.

WHY IT EXISTS

Without the Agent as a distinct, nameable entity, every AI interaction is anonymous. The user cannot build trust, assign responsibility, or reason about which intelligence produced which output. The Agent makes delegation cognitively possible.

EMOTIONAL ROLE

Trust and delegation. An Agent that you have named, configured, and worked with across multiple sessions earns a kind of professional familiarity — the quiet confidence of knowing what a specialist will and won't do well.

WHAT BREAKS WITHOUT IT

The user cannot delegate — they can only prompt. They cannot reason about which intelligence produced which result. The product collapses into a single-model chatbot with an elaborate interface.

User-visible	Always — Agent identity is always displayed on runs and artifacts.
Persistent	Yes — configuration persists across sessions and projects.
Auditable	Yes — configuration changes, capability additions, tool grants.

Memory

WHAT IT IS

The structured, visible, editable knowledge state of a Project. Memory is not a conversation history. It is a set of named facts, decisions, constraints, and context that the system maintains and that gets scoped into agent runs. Four zones: workspace-level facts, project-level context, decision records, ephemeral session context.

WHY IT EXISTS

Without visible, editable Memory, the system's intelligence is invisible and unverifiable. The user cannot know what the agents know. They cannot correct a false assumption. Trust requires transparency, and transparency requires that the knowledge state be a first-class, inspectable entity — not a black box.

EMOTIONAL ROLE

Continuity and groundedness. Memory is what makes the workspace feel like a place you inhabit rather than a tool you operate. When you open a project after three days, Memory is what says 'here is where you were.'

WHAT BREAKS WITHOUT IT

Every agent run starts cold. Context must be re-established manually every session. The workspace has no continuity. The user cannot verify that agents are reasoning from correct premises. Trust is impossible.

User-visible	Always — Memory is a visible, editable panel, never a hidden system prompt.
Persistent	Yes — indefinitely. Ephemeral session context is explicitly labeled.
Auditable	Yes — every fact addition, edit, and deletion is logged with timestamp.

Thread

WHAT IT IS

The temporal spine of a Project. The Thread is the ordered record of everything that has happened: briefs dispatched, runs completed, artifacts produced, decisions recorded, redirects issued, convergences performed. It contains structured, typed entries — not undifferentiated messages.

WHY IT EXISTS

Without the Thread, a project has state but no history. You can see what is currently true but not how you got there. The Thread is the project's memory of actions, as distinct from Memory which is the project's knowledge of facts.

EMOTIONAL ROLE

Narrative and progress. The Thread is what makes work feel like it is accumulating toward something. Scrolling up through a project thread should feel like reviewing a creative process — the thinking, the iterations, the moments of decision.

WHAT BREAKS WITHOUT IT

No record of how the project reached its current state. Lineage becomes impossible. The return-context feature has no source material. Work feels ephemeral even when technically persistent.

User-visible	Always — the Thread is the primary work surface.
Persistent	Yes — append-only. Entries are never deleted, only archived.
Auditable	Inherently — the Thread is itself the audit trail.

TIER 2 — OPERATIONAL

The product's value is delivered through these. They are how work actually happens.

Brief

WHAT IT IS

A structured intent document that directs an Agent. A Brief has four required fields: Goal (what the agent should produce), Context (what it needs to know that is not in Memory), Constraints (what it must not do), and Output format (what kind of artifact it should produce). A Brief is not a message. It is not a prompt.

WHY IT EXISTS

The Brief is the UX primitive that distinguishes a workspace from a chat interface. The act of filling in a Brief is cognitively different from typing a message: it requires the user to be explicit about intent, context, and expectations. The Brief is also the unit of auditable intent.

EMOTIONAL ROLE

Clarity and intention. Writing a Brief should feel like giving a creative brief to a talented specialist — deliberate, considered, and confident. This is a different emotional register from pressing Enter after a chat message.

WHAT BREAKS WITHOUT IT

Without the Brief as a distinct entity, all user intent is implicit and ephemeral. The audit trail loses its intent anchor. The user's posture degrades from director to prompter.

User-visible	Always — the Brief is shown alongside every Run and Artifact it produced.
Persistent	Yes — Briefs are stored and linked to their Runs.
Auditable	Yes — the most auditable entity in the system. Every Run is traceable to its Brief.

Run

WHAT IT IS

A single execution of an Agent against a Brief, within a defined context scope. A Run has status (pending, running, complete, failed, cancelled, redirected), a start time, a completion time, the model used, the memory scope that was active, and the artifacts it produced.

WHY IT EXISTS

Without the Run as an explicit entity, agent activity is invisible. The user cannot see what is happening, cannot redirect a failing execution, cannot understand the cost or time of a given task, and cannot inspect the conditions under which an artifact was produced.

EMOTIONAL ROLE

Presence and activity. A running Run should feel alive in the interface — a gentle indicator that intelligence is at work on your behalf. The transition from running to complete is one of the most important interaction moments in the product.

WHAT BREAKS WITHOUT IT

Execution is a black box. Redirect has nothing to target. Lineage loses its execution record. The user cannot reason about 'why did this take so long' or 'what was the system doing when it failed.'

User-visible	Yes — status and duration visible in Thread and Orchestration view.
Persistent	Yes — Run records are kept indefinitely for lineage.
Auditable	Yes — model, scope, timestamps, token usage, status transitions.

Artifact

WHAT IT IS

Any discrete, named output produced by a Run that has been evaluated by the user. An Artifact is not the raw output of a model. Raw output becomes an Artifact only after it passes through Review. An Artifact has a type (analysis, code, plan, document, synthesis), a production lineage, and a lifecycle status.

WHY IT EXISTS

Without the Artifact as a distinct, typed, lifecycle-managed entity, all AI output is undifferentiated text. The user cannot curate, promote, reject, or supersede it. The Artifact is the unit of creative output in the workspace.

EMOTIONAL ROLE

Judgment and curation. The moment of reviewing an Artifact is the moment the user exercises editorial authority. Approving an Artifact feels like selecting a take. Rejecting one feels like sending notes back to a collaborator.

WHAT BREAKS WITHOUT IT

The product cannot distinguish between 'output' and 'canonical work.' Convergence has no source material. The quality filter between AI production and canonical project state disappears.

User-visible	Yes — all promoted Artifacts are visible in the Thread.
Persistent	Yes — all Artifacts including rejected ones are retained for lineage.
Auditable	Yes — promotion, rejection, and supersession events with timestamps.

TIER 3 — SYNTHESIS AND TRUST

The product earns long-term, repeated use through these. They are what make it feel like a serious tool.

Diverge

WHAT IT IS

A first-class dispatch operation in which a single Brief (or a set of Brief variants) is sent simultaneously to multiple Agents or multiple configurations of the same Agent, producing parallel independent Runs. Diverge creates an Orchestration context, gives each Run a named lane, and suspends normal Thread progression until a Convergence event resolves it.

WHY IT EXISTS

The most cognitively valuable thing you can do with multiple AI specialists is explore a problem from multiple angles simultaneously and then compare. Without Diverge as a named, structured operation, this requires manual repetition and no UI support for comparison.

EMOTIONAL ROLE

Exploration and confidence. The gesture of triggering a Diverge says 'I don't yet know which direction is right, and I am deliberately creating optionality.' The emotional payoff comes at Convergence.

WHAT BREAKS WITHOUT IT

The product is sequential. All AI work is single-threaded, single-perspective. The orchestration value proposition disappears. The product is more sophisticated than a chatbot but not qualitatively different.

User-visible	Yes — Diverge triggers the Orchestration surface, always visible when active.
Persistent	Yes — the Diverge event is recorded in the Thread.
Auditable	Yes — which agents were dispatched, with which briefs, at what time.

Convergence

WHAT IT IS

The structured editorial operation that resolves a Diverge event. Convergence presents the artifacts from parallel Runs in a comparison view, exposes their differences and agreements, and offers two resolution paths: synthesis or selection. Convergence is never automatic. It always requires a deliberate human choice.

WHY IT EXISTS

Without Convergence as a defined operation, Diverge creates chaos with no resolution mechanism. More importantly: Convergence is where the user exercises authorial authority. It is the moment that most directly expresses the identity of 'editor of intelligence.'

EMOTIONAL ROLE

Synthesis and authorship. The Convergence moment is the most emotionally significant interaction in the product. It is where the user feels like they are doing something that AI cannot do alone — making a judgment about what is true, what is useful, and what should become canonical.

WHAT BREAKS WITHOUT IT

Diverge creates a cognitive dead end. Parallel outputs accumulate with no resolution path. The product's most powerful workflow has no conclusion gesture.

User-visible	Yes — dedicated interaction moment in the Orchestration surface.
Persistent	Yes — the Convergence event, editorial choices, and synthesis artifact are persistent.
Auditable	Yes — which artifacts were selected, what synthesis direction was given, what was discarded.

Decision

WHAT IT IS

An explicit, named record of a choice made within a project. A Decision has: the choice itself, the reasoning behind it, the alternatives considered, the date, and optionally the artifact or run that informed it. Decisions can be created manually or extracted automatically from the Thread.

WHY IT EXISTS

Work produces not just artifacts but choices. The difference between a project that can be resumed by someone else and one that cannot is largely whether the decisions are recorded. Agents briefed with a history of project Decisions make better choices.

EMOTIONAL ROLE

Institutional memory and justification. A Decision record is the answer to 'why does this look the way it does?' It prevents the endless re-litigation of choices that have already been made.

WHAT BREAKS WITHOUT IT

Memory contains facts but not reasoning. Agents can be told what constraints exist but not why. New contributors cannot understand the project's logic without reading the entire Thread.

User-visible	Yes — in the Memory panel, organised by date and topic.
Persistent	Yes.
Auditable	Yes — creation, revision, and reversal events.

Snapshot

WHAT IT IS

A named, restorable capture of a project's complete state at a specific moment. Includes: current Memory state, all Artifact content, project goal, agent configurations, and a Thread position reference. Created automatically before every Deploy and manually on demand. Named, not timestamped.

WHY IT EXISTS

Without Snapshots, deployment is a fear gesture. The user has no safe point to return to if something goes wrong. Without Snapshots, the workspace has no concept of 'versions' of a project's state.

EMOTIONAL ROLE

Safety and reversibility. The existence of a Snapshot should change the user's relationship to risk. This is the same emotional function as git — not to record history for its own sake, but to make forward movement feel safe.

WHAT BREAKS WITHOUT IT

Deploy is irreversible and therefore frightening. Experimentation is inhibited. The workspace feels fragile. Operational trust is absent.

User-visible	Yes — named Snapshots visible in the Deploy surface.
Persistent	Yes — stored indefinitely by default.
Auditable	Yes — creation, restore events, and what state was restored.

Lineage

WHAT IT IS

The traceable provenance chain of any Artifact, surfaced as an expandable annotation. Shows: the Brief that created it, the Agent that executed it, the Memory state at execution, the Run details, any Redirects applied mid-run, and whether it was part of a Diverge event. Not a separate surface — an inline disclosure on Artifacts.

WHY IT EXISTS

Without Lineage, the user cannot understand why an artifact looks the way it does. They cannot assess whether the reasoning was sound, whether the context was correct, or whether the agent was working from a stale memory state.

EMOTIONAL ROLE

Trust and accountability. Lineage answers the question: 'How did this get made?' The ability to trace a result back to its intent, context, and execution is what separates a professional tool from a toy.

WHAT BREAKS WITHOUT IT

The system is opaque. Incorrect results cannot be diagnosed. Trust is faith-based rather than evidence-based. In any serious use case, opacity is a terminal flaw.

User-visible	Yes — as an expandable annotation on Artifacts.
Persistent	Yes — immutable once created.
Auditable	Inherently — Lineage is itself an audit mechanism.

PART II

The Interaction Grammar

Actions are the verbs of the platform. Every action should have a distinct weight — some are light (instant, reversible, low consequence), some are heavy (deliberate, often irreversible, high consequence). The weight of an action should be felt in the interaction, not just described in documentation.

Query	LIGHT	Instant recall. No run created. Like asking a knowledgeable colleague. The lightest interaction in the system.
Review	LIGHT	Judicial and fast. Three distinct states: approve, reject, redirect. Not a rating widget — an editorial act.
Redirect	MEDIUM	Corrective and precise. A margin note to a specialist who is still working. Applied to a Run, not to an Agent.
Promote	MEDIUM	Curatorial and selective. 'This represents the project's state now.' Moves an Artifact from reviewed to canonical.
Brief	MEDIUM	Deliberate and structured. Commissioning a specialist. Requires explicit intent, context, and constraints. Never a casual action.
Dispatch	MEDIUM	Committed and one-way. Once dispatched, a Run begins. Should carry a sense of finality.
Snapshot	HEAVY	Intentional checkpoint. Requires naming — not a timestamp. A deliberate save before significant change.
Diverge	HEAVY	Expansive. Creates an Orchestration context. Multiple Runs begin simultaneously. The fan-out gesture.
Converge	HEAVY	Authorial. The most significant gesture in the orchestration workflow. Editorial authority explicitly asserted.
Deploy	CRITICAL	Consequential. Requires a named Snapshot, health-check acknowledgment, and explicit confirmation. Never one click from a resting state.

Secondary verbs (system-initiated)

Extract — the system extracts a Decision from Thread content and presents it for user confirmation. System-initiated, user-confirmed.

Synthesize — during Convergence, the system produces a unified Artifact from multiple inputs per the user's editorial direction. System-executed, user-directed.

Recall — the system surfaces relevant Memory in response to a Query. System-executed, triggered by user.

Expire — session context that is explicitly ephemeral is expired at session end. System-initiated, user-informed.

PART III

Surface Allocation

Not every primitive deserves its own page. The surface allocation decision is one of the most consequential architectural choices in the product.

Primitives that deserve dedicated surfaces

Project / Thread

The primary work surface. Everything that happens within a project happens here. Must be the most refined surface in the product.

Orchestration

Appears as an overlay on the Thread when a Diverge event is active. Not a separate page — a contextual overlay. Disappears when Convergence is complete.

Memory panel

A persistent panel within the Project surface. Always accessible. Never hidden. Divided into four zones. Editable in place.

Agent Studio

A dedicated surface for configuring Agents. Separate from the Thread because Agent configuration is a deliberate, infrequent act that should have full attention.

Deploy / Snapshot

A dedicated surface for operational state: Snapshot history, Deploy history, health status, environment configuration. Accessed intentionally, not ambient.

Command palette

Accessible from anywhere via keyboard shortcut. Spans surfaces. The interface between the user's intent and the entire system.

Primitives that live within surfaces

Brief — lives in the Composer at the bottom of the Thread. Does not have its own page. Run — visible in the Thread as a typed entry. Not a separate page. Artifact — lives in the Thread, expandable inline. Decision — lives in the Memory panel. Lineage — an expandable disclosure on any Artifact. Never a separate page. Query — lives in the Command palette and the Memory panel search.

PARTS IV & V

Persistence and Auditability

Persistent (retained indefinitely by default)

Project, Agent configuration, Memory (all zones except ephemeral), Thread (append-only), all Artifacts including rejected ones, all Runs, Briefs, Decisions, Snapshots, Lineage records, Convergence events, Diverge events, Deploy history.

The rule: anything a user might need to answer 'how did this get here?' must be persistent.

Ephemeral (expires at session end, explicitly labeled)

Session context in Memory (the fourth zone), draft Briefs that have never been dispatched, intermediate synthesis states during Convergence before the user confirms.

The rule: anything that would be confusing if it persisted must be ephemeral and explicitly labeled. The Memory panel's ephemeral zone must look different from the persistent zones. Every ephemeral entity must be visually labeled as such.

Auditable primitives

Brief — What was asked. Once dispatched, never edited — only superseded by a new Brief. The intent record is sacred.

Run — What was executed. Model, scope, timestamps, status transitions, token usage. Immutable.

Decision — What was decided and why. Creation, revision, and reversal events with timestamps.

Converge event — What was synthesized, what was discarded, what editorial direction was given.

Deploy — What was deployed, from what Snapshot, with what health-check outcome. Non-repudiation for operational events.

Memory edit — Every addition, edit, and deletion to Memory is logged. Prevents silent context drift.

PART VI

The Emotional Core

Three primitives define the product's soul. Without these three feeling right — not just functioning, but feeling right — no amount of polish in the remaining primitives will save the product's identity.

Memory

If the workspace does not feel like it knows your project, it does not feel like a workspace. It feels like a slightly sophisticated chatbot. Memory is the primitive that answers 'this place knows where I was.' It is what separates a tool you use once from a workspace you inhabit over time.

EMOTIONAL TEST

When a user opens a project they haven't touched in four days, do they feel oriented or disoriented? Oriented is right. Disoriented means Memory has failed, regardless of whether it is technically functioning.

Brief

If directing an Agent feels like chatting with a bot, the product has failed its core identity claim. The Brief is the primitive that makes the user feel like a director rather than a prompter. The act of constructing a Brief is the act of taking a professional posture toward AI work.

EMOTIONAL TEST

Does filling in a Brief feel like preparing to commission good work, or does it feel like typing a prompt? The former is right.

Convergence

The moment a user resolves a Diverge event — reviewing parallel outputs, making editorial choices, directing synthesis — is the moment they most clearly feel their authorial authority. This is the interaction that most definitively answers 'this is not a chatbot' and 'I am not just a reader of AI output.'

EMOTIONAL TEST

Does the user feel like a director making a call, or like a user selecting an answer from a multiple-choice question? The former is right. The latter is a chatbot with extra steps.

PART VII

The Coherence Rules

These rules govern all future product decisions. Any proposed feature or change must pass them.

Rule 1 — Entity fidelity

No new concept may be introduced that maps to a combination of existing primitives without a demonstrated reason why the combination is insufficient. Prevents abstraction inflation.

Rule 2 — The verb test

Every user-facing action must have a named verb from the interaction grammar. If a proposed interaction cannot be described by an existing verb, either the interaction is wrong or a new verb must be justified at the level of a primitive — not a feature.

Rule 3 — Memory visibility

No information that informs an agent run may be hidden from the user. Memory must be inspectable and editable. No hidden system prompts that the user cannot see or modify.

Rule 4 — Lineage completeness

Any Artifact that appears in the Thread must have a complete Lineage. If an Artifact cannot be traced to a Brief, an Agent, and a Run, it should not appear in the Thread.

Rule 5 — The chatbot test

Before any new feature is designed, ask: does this make the product feel more like a chatbot or less? If more, reject or fundamentally redesign. The product's identity is defined by what it refuses to become.

Rule 6 — The weight gradient

No two verbs should have the same interaction weight. Light verbs should feel instant. Heavy verbs should require deliberate gesture. If Deploy can be triggered accidentally, the design has failed.

Rule 7 — Ephemeral labeling

Anything ephemeral must be labeled as such in the interface, ambient and always. The user must never discover that something they relied on has expired.

This document is a constitution, not a backlog. It defines what the platform is, not what it will ship next. Amendments require the same level of argument applied here — structural justification, not feature rationale.